

Enhancing Readability of Automatically Generated Unit Tests: Leveraging LLMs for Meaningful Identifier Naming

Jason Liu

Master Student Software Engineering
University of Antwerp
Antwerp, Belgium
jason.liu@student.uantwerpen.be

Abstract—

Index Terms—Large Language Models, Testing, Readability, Identifiers, Code refactor, Unit tests, Automated test generation, Java, Variables, Method names

I. INTRODUCTION

Unit tests play a central role in modern software development. They verify that individual units of code behave as intended, serve as a safety net against regressions, and act as executable documentation for engineers studying an unfamiliar system [1], [2]. Beyond correctness, tests must also be *readable*: a test that cannot be understood quickly cannot be trusted, debugged, or extended [3]. Readable tests reduce the cognitive load of fault localisation and help developers understand the intended behavior of the system under test without consulting production code [4].

Writing such tests by hand is labor-intensive, which has driven wide adoption of automated test generation tools such as EvoSuite [5] and Randoop [6], [7]. EvoSuite applies genetic algorithms to maximise a coverage criterion such as branch coverage [8]; Randoop uses feedback-directed random testing to generate and filter sequences of method calls [7]. Both tools operate purely on structural and execution properties and have no awareness of semantic meaning, so the tests they produce are populated with generic identifiers such as `var0`, `int0`, and `test1` [9], [10].

This readability deficit has measurable consequences. Butler et al. showed that poor identifier naming degrades code comprehension and maintainability [11], and Lin et al. found that identifier quality in test code is substantially lower than in production code, with only 61.6% of test identifiers rated acceptable compared with 92.5% in production code [12]. Winkler et al.’s systematic mapping study confirmed that uninformative identifiers are one of the primary factors impeding comprehension of automatically generated tests, and that no single metric adequately captures all dimensions of test readability [13].

Prior work has approached the problem from several angles. DeepTC-Enhancer [3] combined template-based and deep learning methods to improve test readability, though 48% of

suggested variable names only *somewhat* conveyed the intended meaning, and the evaluation relied on a small human study that cannot scale to large test suites. De Keersmaecker [14] evaluated GGNN and RefBERT adapted for generated tests and found that neither achieved consistent, concise naming across full test files. Most recently, Biagiola et al. [4] used LLMs to rename identifiers in EvoSuite-generated tests and reported readability on par with developer-written tests in a study with ten professional developers, a promising result, but one that relies on closed proprietary models and does not evaluate fine-tuned open-source alternatives.

A complementary challenge is *evaluation*: measuring whether a renaming actually improves readability. Reference-based metrics such as BLEU [15] are unsuitable because multiple valid names can exist for the same variable, so a correct rename may score near zero against a single reference [16], [17]. The LLM-as-a-Judge paradigm addresses this by using language models as evaluators that assess semantic quality against explicit rubrics, achieving over 80% agreement with human preferences on code-related tasks [18], [19]. Single-judge evaluation is, however, susceptible to model-specific biases such as verbosity preference and position sensitivity [18]; aggregating scores across an ensemble of diverse judge models has been shown to reduce this bias and improve alignment with human judgment [20], [21].

These results suggest that LLMs are a promising vehicle for identifier renaming and for evaluating renamed output at scale, yet key questions remain open. Existing work does not compare fine-tuned open-source models against untuned baselines in this setting, does not provide a configurable and reproducible readability grading framework, and does not examine whether LLM-based readability scores are consistent with quantitative naming metrics such as F1, a tension that must be understood before either metric can be trusted as a ground truth.

This paper addresses the following research questions:

- **RQ1:** Does an LLM-based renaming pipeline outperform the non-LLM baselines (GGNN and RefBERT) on identifier naming quality in automatically generated Java tests?
- **RQ2:** How does the amount of fine-tuning affect naming quality across checkpoints (15%, 25%, 50%, 75%, 100%)?

- **RQ3:** Does token-level improvement (F1) correlate with perceived readability as measured by the LLM readability score?

II. BACKGROUND

A. Automated Test Generation

EvoSuite [5] evolves a population of test cases using a genetic algorithm that maximises a coverage criterion such as branch coverage. Randoop [6], [7] generates sequences of method calls incrementally, using execution feedback to discard failing sequences and retain those that extend coverage. Both tools operate purely on structural and execution properties, with no awareness of semantic meaning, so the identifiers they emit are generic and uninformative: local variables receive names such as `double0` or `calculator0`, and test methods receive names such as `test0` or `test42` [9], [10]. Examples are shown in Listings 6 and 7.

B. Identifier Quality and Test Readability

A high-quality identifier satisfies three criteria [14]: *consistency* (no synonyms or homonyms across the codebase), *correctness* (the name corresponds to the concept it represents), and *conciseness* (the name is no more general than the concept warrants). Butler et al. demonstrated empirically that poor identifier naming degrades a developer’s ability to understand and maintain code [11], and Lin et al. found that test code identifiers are substantially lower quality than production code identifiers, with only 61.6% rated acceptable compared to 92.5% for production code [12]. Test method names carry an additional communicative burden: the established JUnit convention encodes the scenario and expected outcome directly in the name using a pattern such as `methodName_stateUnderTest_expectedBehavior` [1]. When generated tests use names like `test42`, neither purpose is served.

Of the three criteria, *consistency* deserves particular attention in the context of automatically generated tests. A name that is individually readable but inconsistent with the surrounding codebase’s naming conventions may be harder to work with than a generic name that follows a predictable pattern, because developers rely on naming regularity to navigate and reason about code quickly [11], [22]. Winkler et al.’s systematic mapping study confirmed that identifier consistency is among the primary factors influencing comprehension of test code, and that no single metric adequately captures all dimensions of test readability [13].

C. Large Language Models for Code

LLMs are transformer-based models pre-trained on large corpora of code and natural language [23]. This pre-training gives them an implicit understanding of naming conventions and the relationship between code structure and semantic intent, which task-specific models had to be explicitly engineered to capture [24], [25]. Code-specialised models push this further: DeepSeek-Coder is trained on 2 trillion tokens of which 87% is source code [26], while Qwen2.5-Coder is pre-trained on

over 5.5 trillion tokens spanning 92 programming languages and achieves state-of-the-art performance across more than 10 code benchmarks [27]. Both models have been shown to perform rename refactorings successfully, demonstrating that they have internalised sufficient naming knowledge from their training data [28]. LLM performance on code tasks is significantly affected by the completeness of contextual information available during inference [29], which motivates providing the full function body alongside the identifier list. Few-shot prompting improves output reliability [23], and when a response is malformed the Reflexion retry strategy feeds the failed response back to the model with a concrete error description, substantially increasing the probability of a valid response on the next attempt [30], [31].

D. Fine-Tuning LLMs

Pre-trained LLMs can be adapted to a specific task by continuing training on a small, task-relevant dataset, a process known as fine-tuning [32], [33]. Fine-tuning allows the model to specialize its learned representations for the target domain while retaining the broad knowledge acquired during pre-training. In the context of identifier renaming for generated tests, fine-tuning on human-written test code that has been obfuscated to simulate generated output exposes the model to the specific structural patterns and naming conventions of the target domain, potentially improving both accuracy and consistency of suggestions compared to an untuned baseline. The extent to which fine-tuning improves naming quality, measured both by token-level metrics and by LLM-based readability scoring, is one of the central empirical questions of this work.

E. LLM-as-a-Judge Evaluation

Reference-based metrics such as BLEU are unsuitable for evaluating identifier naming because multiple valid names may exist for the same variable, so a correct rename can score near zero against a single reference [15]–[17]. The LLM-as-a-Judge paradigm addresses this by using LLMs as evaluators that assess semantic quality directly against explicit rubrics, achieving over 80% agreement with human preferences [18]. Structured rubrics with fine-grained, independently assessable criteria produce more stable scores than vague numeric scales [34], and aggregating across an ensemble of diverse judge models further reduces individual model bias [20].

III. RELATED WORK

A. Identifier Renaming Without LLMs

Early approaches to automated identifier renaming treated source code as a sequence of tokens and applied statistical language models. Mastropaolo et al. conducted a large-scale empirical study comparing an n-gram model, a T5 model, and a BERT-based model (CugLM) for variable renaming in production code, finding that CugLM achieved 63.5% complete match but that all approaches struggled when multiple valid names existed for the same variable [17]. Context2Name [35] used a sequence autoencoder over local token windows to

infer variable names from minified JavaScript, achieving 47.5% exact match, but its dependence on surrounding tokens makes it unsuitable for the fully obfuscated identifiers in generated tests.

Structure-aware approaches address this by operating on graph representations of code rather than raw token sequences. Allamanis et al. applied Gated Graph Neural Networks (GGNNs) to Abstract Syntax Trees augmented with dataflow edges, achieving 32.9% subtoken-level precision on production Java code [24]. De Keersmaeker adapted both GGNN and RefBERT for automatically generated tests [14]: RefBERT outperformed GGNN in token-level accuracy, but neither approach produced consistent or concise names across full test files, and both tended to oversimplify concepts and ignore the test scenario in the suggested name. These results of De Keersmaeker serve as the direct baselines for our work.

B. LLM-Based Test Improvement

DeepTC-Enhancer [3] was the first work to directly target readability of automatically generated tests, combining template-based test summarization with deep learning to suggest variable names. A user study reported that 48% of suggestions fully conveyed the intended meaning and 35% somewhat conveyed it, leaving substantial room for improvement. Daka et al. showed that even rule-based synthesis of test method names from coverage criteria can achieve 53% developer agreement, establishing that method naming is a tractable sub-problem [1].

Biagiola et al. proposed the most closely related prior work: they use LLMs to rename identifiers and test method names in EvoSuite-generated tests while explicitly preserving coverage semantics [4]. Their evaluation across nine industrial and open-source LLMs showed that the readability improvements are semantically stable across repetitions, and a human study with ten professional developers found LLM-improved tests as readable as developer-written ones. Our work differs in several important ways. First, Biagiola et al. evaluate a mix of proprietary and open-source models primarily through external APIs; our pipeline targets locally served open-source models loaded via Hugging Face Transformers, which are stateless and process each function in a single, self-contained prompt. Second, because our pipeline communicates through a model-agnostic API layer, it is not tied to any specific model or provider. Third, we evaluate fine-tuned open-source models alongside untuned ones, a comparison Biagiola et al. do not include. Fourth, our pipeline operates on identifiers extracted from any well-formed Java test method regardless of which generator produced it. Finally, we contribute a configurable, reproducible readability grading framework that Biagiola et al. do not provide.

C. Readability Evaluation of Test Code

Scalabrino et al. proposed a comprehensive readability model for source code that combines structural features such as line length and nesting depth with textual features such as identifier quality and comment density [36]. Their findings show that

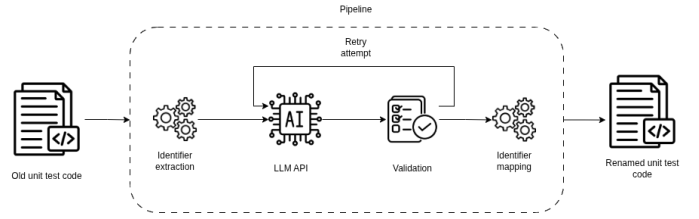


Fig. 1. Renaming pipeline workflow.

automatically generated tests are doubly penalised: they lack comments entirely, and their identifiers are of poor quality, making structural metrics alone insufficient to capture their readability. Takebayashi et al. studied the role of assertion messages and found that expressive assertions are an important but often overlooked readability factor in test code [37]. Winkler et al.’s systematic mapping study confirmed that no single metric captures all dimensions of test readability, and that human studies remain the gold standard but are not scalable [13].

These limitations motivate the LLM-as-a-Judge evaluation design of our grading tool: it directly assesses semantic quality along multiple independently weighted criteria, avoiding the ceiling imposed by reference-based metrics while remaining fully automated and reproducible.

IV. APPROACH

We discuss both the renaming pipeline and the readability grading tool.

A. Pipeline

The renaming pipeline, illustrated in Figure 1, consists of two stages: identifier extraction and LLM-based renaming. In the extraction stage, each test method is isolated from its enclosing class and all identifiers, both variable names and method names, are collected along with their kind. In the renaming stage, the complete obfuscated method body and the extracted identifier list are passed together to the LLM as a single prompt, so that the model has full syntactic context when suggesting new names [24], [29]. The pipeline processes test methods one by one [14], keeping each call self-contained and stateless.

Since LLM outputs can be malformed or structurally invalid, the pipeline supports a configurable retry mechanism: when a response fails schema validation, the failed output and the specific error reason are fed back to the model in a follow-up prompt [30], [31], [38]. Upon a successful response, each suggested name is mapped back to its original location in the function body and the renamed method is reconstructed.

1) *Prompt Design*: The pipeline communicates with the LLM through three prompt templates: a system instruction, a user prompt, and a retry prompt.

The user prompt provides the complete obfuscated test method as context alongside the identifier list. This is motivated by the established finding that identifier meaning is inseparable from its surrounding code structure [24]. Prior

work has further shown that LLM performance on code tasks is significantly affected by the quality and completeness of contextual information available during inference [29], making full function body provision essential for semantically accurate renaming suggestions.

Each identifier is tagged with its kind: (method) or (variable). Test method names and variable names follow distinct conventions — method names should describe the test scenario in camelCase starting with `test` [1], while variable names should use concise nouns reflecting the value’s role. The kind tag encodes this distinction directly in the prompt, operationalizing the consistency and conciseness criteria for identifier quality [14].

The prompt constrains the model to respond with a single JSON object mapping original names to new names, including a one-shot example of the expected format [23]. This structured output constraint enables deterministic schema validation, which in turn drives the retry mechanism [30], [31], [38]. The maximum number of retries is configurable. The full prompt templates are provided in Appendix A.

2) *Tuned Models*: To investigate whether specialization on test code improves renaming quality, we fine-tune a pre-trained LLM on a dataset of obfuscated Java test methods paired with their original identifier names. Fine-tuning is preferred over training from scratch because it allows the model to retain the broad syntactic and semantic knowledge of code acquired during pre-training while specializing its representations for the structural patterns of automatically generated tests [32], [33].

Full fine-tuning of a modern LLM is prohibitively expensive on a single GPU: a 7B parameter model requires roughly 84 GB in 16-bit precision [39]. We therefore apply QLoRA [39], which keeps the base model weights frozen and quantized to 4-bit NormalFloat (NF4) precision while injecting trainable low-rank adapter matrices into each Transformer layer. The adapter parameterization follows LoRA [40]: for a weight matrix $W \in \mathbb{R}^{d \times k}$, the update is decomposed as $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$, reducing trainable parameters by up to four orders of magnitude compared to full fine-tuning. Adapters are injected into all seven projection matrices of each Transformer block (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), which has been shown to improve downstream performance over targeting attention projections alone [40]. Optimization uses paged AdamW in 8-bit precision with a cosine learning rate schedule and a 3% linear warmup.

The model is trained as a causal language model on sequences composed of the prompt followed by the completion, where the completion is the JSON mapping of obfuscated names to original names. Prompt tokens are masked with the ignore index in the labels tensor so that the cross-entropy loss is computed exclusively over the predicted identifier names, ensuring the model learns the renaming task rather than reproducing its own input. Samples whose total token length exceeds the maximum context window are discarded rather than truncated, to avoid training on partial completions

that would provide a misleading learning signal.

To reduce overfitting on the fine-tuning dataset, we apply NEFTune [41], which adds uniform random noise to the embedding vectors during each forward pass. The training run saves adapter snapshots at 15%, 25%, 50%, 75%, and 100% of total training steps, enabling analysis of how naming quality evolves with the amount of fine-tuning.

B. Readability Grading Tool

To evaluate the quality of renamed test code, we developed *CodeReader*, an open-source LLM-based readability grading tool available on PyPI [42]. The tool implements the LLM-as-a-Judge paradigm [18], in which one or more LLMs act as evaluators, assessing the readability of a given test method against a configurable set of domain-specific rules.

The choice of LLM-based evaluation over reference-based metrics such as BLEU is motivated by a fundamental limitation: lexical similarity metrics cannot capture whether an identifier name is semantically appropriate for its context [15], [16], [43]. LLM judges address this by evaluating semantic quality directly, and have been shown to align with human preferences at over 80% agreement [18].

CodeReader is configured through a YAML file that defines a set of named evaluation rules. In our configuration, these rules cover: clarity of variable and method names, behavioral specificity of the test scenario, consistency with JUnit naming conventions, alignment between identifier names and actual test logic, assertion quality, and the presence of unexplained magic numbers. Each rule is expressed as a concrete, actionable criterion rather than a vague dimension, which directly follows the G-Eval recommendation [34].

To mitigate the variance inherent in single-model evaluation, *CodeReader* supports running multiple LLM judges simultaneously with configurable per-model weights. The final score is a weighted aggregation across all active models. In our configuration, three models are used: DeepSeek-Coder 6.7B (weight 1.4), Qwen3.5 9B (weight 1.25), and Phi-3 Medium (weight 0.75), all served locally via Ollama. The use of an ensemble of judges reduces the impact of individual model bias and improves scoring stability [20]. Full configuration details are provided in Appendix A.

V. EXPERIMENTAL SETUP

A. Dataset

The selected dataset for this study is *methods2test* [44], which is the largest publicly available dataset of its kind, containing approximately 780,000 instances. Each entry consists of a mapping between a focal method and its corresponding unit test case, along with metadata such as the class name, package, and source repository.

A minor reduction of the dataset was necessary prior to use: several entries were found to be incomplete or malformed (unclosed braces, truncated functions). The test cases were manually obfuscated such that each function name was renamed to `func_n` and each variable name to `var_n`, as illustrated in Listing 8. Each obfuscated test case was paired with its

TABLE I
SUMMARY OF EVALUATED MODEL VARIANTS.

Model	Params	Fine-tuned	Ckpt
Qwen2.5-Coder (base)	7B	No	—
Qwen2.5-Coder (15%)	7B	Yes	15%
Qwen2.5-Coder (25%)	7B	Yes	25%
Qwen2.5-Coder (50%)	7B	Yes	50%
Qwen2.5-Coder (75%)	7B	Yes	75%
Qwen2.5-Coder (100%)	7B	Yes	100%
GPT-5 mini	n/a	No	—

original version and stored as a JSONL object, as shown in Listing 9.

The evaluation subset consists of 100 randomly sampled test methods drawn with a fixed seed from the held-out test split of methods2test [44]. This sample size is grounded in a prospective power analysis using the Wilcoxon matched-pairs effect-size r [45], [46]. Full statistical details are provided in Appendix A.

B. Models Evaluated

We evaluate two models: Qwen2.5-Coder 7B as the fine-tuned open-source model, and GPT-5 mini as a zero-shot closed-source reference baseline.

Qwen2.5-Coder 7B [27] is a code-specialized open-source model pre-trained on 5.5 trillion tokens across 92 programming languages. Its code specialization and permissive licence make it a practical base for fine-tuning on our obfuscated test-code dataset using QLoRA [39]. Adapter snapshots are saved at 15%, 25%, 50%, 75%, and 100% of training steps, yielding five fine-tuned variants alongside the untuned base model.

GPT-5 mini [47] is a cost-efficient model from OpenAI’s GPT-5 family with a 400K-token context window and native structured-output support. It is evaluated zero-shot as a proprietary upper-bound baseline.

C. Baselines

We compare against the GGNN [24] and RefBERT [48] techniques as adapted for generated test code by De Keersmaecker [14]. Both are re-evaluated on our subset to ensure a fair comparison under identical data conditions.

D. Metrics

1) *Token-level Metrics*: To enable comparison with the GGNN and RefBERT baselines, we adopt the sub-token-level evaluation protocol established by Allamanis et al. [24] and widely used in code identifier naming research [48], [49]. Since identifiers are composed of sub-tokens (e.g., `getUserList` splits into `{get, user, list}`), precision measures how many predicted sub-tokens are correct, recall measures how many expected sub-tokens were produced, and F1 is their harmonic mean.

Beyond sub-token overlap, exact match measures whether all sub-tokens of a predicted identifier are fully correct. We report both the ordered variant, where sub-token order must match, and the unordered variant, which treats sub-tokens as

a set [48]. Character Error Rate (CER) captures the relative number of character-level edit operations needed to transform a prediction into the reference [24].

2) *LLM Readability Score*: Token-level metrics evaluate each identifier in isolation against a fixed reference label. This is insufficient for our setting: multiple valid names can describe the same concept, and the collective readability of all renamed identifiers in a test method matters more than individual sub-token overlap [36].

We adopt an LLM-as-judge approach [18], which has been shown to correlate with human preference at over 80% agreement. Applied to code evaluation specifically, ICE-Score [19] demonstrated that instructing LLMs to score generated code achieves superior correlation with human judgement compared to token-matching metrics such as BLEU. Recent work has further shown that LLMs can serve as scalable and human-aligned readability evaluators for both functional code and unit tests specifically [50].

The judge scores each test method on a scale of 0 to 100 based on four weighted criteria: identifier naming clarity and consistency (weight 0.75), behavioral specificity of the test method name (0.15), and assertion quality (0.10). The score is averaged across all test methods in the benchmark (`llm_score_avg`). A weighted average (`llm_score_wavg`) is also computed, weighting each test by the number of identifiers it contains to give more weight to complex tests.

VI. RESULTS & DISCUSSION

A. RQ1: LLM Pipeline vs. Non-LLM Baselines

Table II reports the token-level metrics for both non-LLM baselines and all Qwen2.5-Coder checkpoints evaluated on the same subset. T1 (GGNN) performs poorly on our subset, achieving an F1 of 0.019 and a CER of 143.9%, indicating that the model almost entirely fails to produce correct subtokens. This result is consistent with De Keersmaecker’s observation that GGNN struggles with obfuscated test code due to the absence of the method under test in the graph representation [14]. T2 (RefBERT) performs considerably better at F1 = 0.308, though its CER of 92.6% still indicates substantial character-level divergence.

The untuned Qwen2.5-Coder baseline (0%) already far surpasses both baselines, achieving F1 = 0.760, precision = 0.714, and recall = 0.817, with a CER of 32.0%. All fine-tuned checkpoints improve further, reaching F1 scores between 0.834 and 0.839. These results confirm that the LLM-based pipeline provides a large and consistent improvement over both non-LLM baselines, directly answering RQ1 affirmatively.

B. RQ2: Effect of Fine-Tuning Amount

The results in Table III reveal a clear and immediate effect of fine-tuning on token-level metrics: F1 increases sharply from 0.760 at the untuned baseline (0%) to 0.834 at the first checkpoint (15%), after which it plateaus and remains stable across all subsequent checkpoints. This suggests that the majority of the fine-tuning benefit is captured very early in

TABLE II
RQ1: TOKEN-LEVEL METRICS FOR BASELINES AND QWEN2.5-CODER CHECKPOINTS.

Model	P	R	F1	CER (%)
T1: GGNN	0.026	0.015	0.019	143.9
T2: RefBERT	0.320	0.306	0.308	92.6
Qwen (0% — untuned)	0.714	0.817	0.760	32.0
Qwen (15%)	0.848	0.825	0.834	17.5
Qwen (25%)	0.853	0.831	0.839	16.9
Qwen (50%)	0.849	0.828	0.836	17.5
Qwen (75%)	0.853	0.824	0.836	17.2
Qwen (100%)	0.849	0.826	0.835	17.3

TABLE III
QWEN2.5-CODER RESULTS PER CHECKPOINT. THE 0% ROW IS THE UNTUNED BASE MODEL.

Ckpt	P	R	F1	LLM	LLM (w)
0%	0.714	0.817	0.760	69.96	71.97
15%	0.848	0.825	0.834	65.11	67.47
25%	0.853	0.831	0.839	64.76	67.12
50%	0.849	0.828	0.836	65.83	68.47
75%	0.853	0.824	0.836	63.60	66.07
100%	0.849	0.826	0.835	65.60	68.19

training, and that additional training steps beyond 15% yield diminishing returns on token-level naming accuracy.

A secondary finding concerns schema compliance. The untuned baseline produced 7 `ExtraKeyError` responses across 100 files. All fine-tuned checkpoints from 15% onward produced zero schema errors, indicating that fine-tuning substantially improves instruction following.

C. RQ3: F1 vs. LLM Scoring

Looking at Table III, the LLM readability score tells a different story. The untuned baseline (0%) achieves the highest readability score of all variants (avg: 69.96), which then drops at 15% (65.11) and remains consistently lower across all fine-tuned checkpoints. A plausible explanation is that the reference labels in the `methods2test` dataset reflect the naming conventions of real-world Java codebases, which do not always align with the explicit readability criteria defined in the `CodeReader` rubric. Fine-tuning causes the model to converge towards dataset-specific naming patterns, which may be consistent with the codebase from which each test originates but do not necessarily satisfy general readability principles such as behavioral specificity or semantic alignment with test logic [12], [36]. The untuned model, unconstrained by dataset patterns, produces more varied and sometimes more descriptive names that the judge rewards more highly, even at the cost of lower token-level overlap with the reference.

This tension points to a fundamental limitation of reference-based evaluation for identifier naming: a model can score well on F1 by learning dataset-specific naming conventions without producing names that are inherently more readable, a concern previously raised by Mastropaolo et al. [17] and Evtikhiev et al. [16].

D. Discussion

The three research questions together reveal a nuanced picture of LLM-based identifier renaming. RQ1 establishes that even an untuned LLM substantially outperforms dedicated non-LLM approaches, suggesting that the broad code understanding acquired during pre-training is sufficient to transfer meaningfully to the renaming task without task-specific adaptation. RQ2 shows that fine-tuning provides an immediate and significant token-level gain but saturates rapidly, with 15% of training steps capturing most of the benefit, a practically important finding given the computational cost of fine-tuning. The most consequential finding emerges from RQ3: fine-tuning improves F1 while simultaneously reducing the LLM readability score. This divergence suggests that token-level accuracy and holistic readability are measuring fundamentally different properties. This has direct implications for how future work should evaluate identifier renaming tools: neither F1 alone nor an LLM readability score alone is a sufficient ground truth, and the two metrics should be reported together.

VII. THREATS TO VALIDITY

This section discusses the potential threats to the validity of the study following the categorization of Wohlin et al. [51], considering construct, internal, external, and conclusion validity.

A. Construct Validity

The primary construct validity threat concerns the LLM readability score. While the scoring criteria are grounded in established identifier quality research [12], [36], the score is produced by an ensemble of smaller open-source judge models rather than human evaluators, and may not fully capture what developers consider readable in practice. Three interrelated limitations compound this threat. First, the judge models are generative and therefore non-deterministic. Second, the choice of judge models may itself introduce bias. Third, different judge model selections may produce different relative rankings.

B. Internal Validity

The dataset is constructed by obfuscating human-written tests rather than using genuinely auto-generated tests. While this simulates the naming patterns of tools like `EvoSuite` [5], the structural characteristics may differ. Additionally, the `methods2test` dataset [44] contains known identifier quality issues [12], meaning the oracle is not a perfect upper bound.

C. External Validity

The evaluation is limited to 100 randomly sampled Java test methods from `methods2test` [44]. Results may not generalize to other programming languages, other test generators, or test methods from different domains.

D. Conclusion Validity

The 100-sample subset is statistically adequate for all primary large-effect comparisons as demonstrated in Appendix A, but the renamed vs. oracle comparison remains underpowered for some checkpoints. Conclusions about the absolute quality gap between renamed and human-written output should therefore be treated as indicative rather than definitive.

VIII. CONCLUSION

REFERENCES

- [1] E. Daka, J. M. Rojas, and G. Fraser, "Generating unit tests with descriptive names or: Would you name your children thing1 and thing2?" in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. New York, NY, USA: ACM, 2017, pp. 57–67.
- [2] S. Demeyer, S. Ducasse, and O. Nierstrasz, *Object-Oriented Reengineering Patterns*. Square Bracket Associates, 2009, available at <http://scg.unibe.ch/oorp/>.
- [3] D. Roy, Z. Zhang, M. Ma, V. Arnaoudova, A. Panichella, S. Panichella, D. Gonzalez, and M. Mirakhorli, "DeepTC-Enhancer: Improving the readability of automatically generated tests," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. New York, NY, USA: ACM, 2020.
- [4] M. Biagiola, G. Ghisloti, and P. Tonella, "Improving the readability of automatically generated tests using large language models," in *Proceedings of the 2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. Naples, Italy: IEEE, 2025.
- [5] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. New York, NY, USA: ACM, 2011, pp. 416–419.
- [6] C. Pacheco and M. D. Ernst, "Randoop: Automatic unit test generation for Java," <https://randoop.github.io/randoop/>, 2007, accessed: 2024.
- [7] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-directed random test generation," in *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. Minneapolis, MN, USA: IEEE Computer Society, 2007, pp. 75–84.
- [8] P. McMinn, "Search-based software testing: Past, present and future," in *Proceedings of the 4th International Workshop on Search-Based Software Testing (SBST)*. Berlin, Germany: IEEE, 2011, pp. 153–163.
- [9] G. Fraser and A. Arcuri, "EvoSuite: On the challenges of test case generation in the real world," in *Proceedings of the 6th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2013, pp. 362–369.
- [10] J. Wang, B. Suleiman, and M. J. Alibasa, "Automated unit test case generation: A systematic literature review," 2025. [Online]. Available: <https://arxiv.org/abs/2504.20357>
- [11] S. Butler, M. Wermelinger, Y. Yu, and H. Sharp, "Exploring the influence of identifier names on code quality: An empirical study," in *Proceedings of the 14th European Conference on Software Maintenance and Reengineering (CSMR)*. Madrid, Spain: IEEE, 2010, pp. 156–165.
- [12] B. Lin, C. Nagy, G. Bavota, A. Marcus, and M. Lanza, "On the quality of identifiers in test code," in *2019 19th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2019, pp. 204–215.
- [13] D. Winkler, P. Urbanke, and R. Ramler, "Investigating the readability of test code: Combining scientific and practical views," *Empirical Software Engineering*, vol. 29, no. 2, p. 53, 2024.
- [14] A. De Keersmaeker, "Enhancing test code understandability with machine learning-based identifier naming," Master's Thesis, University of Antwerp, 2024, supervisor: Prof. dr. Serge Demeyer; Supervising assistant: Dr. Mutlu Beyazit.
- [15] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. Philadelphia, PA, USA: Association for Computational Linguistics, 2002, pp. 311–318. [Online]. Available: <https://aclanthology.org/P02-1040/>
- [16] M. Evtiukhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, "Out of the BLEU: How should we assess quality of the code generation models?" *Journal of Systems and Software*, vol. 203, p. 111741, 2023.
- [17] A. Mastropaolo, E. Aghajani, L. Pascarella, and G. Bavota, "Automated variable renaming: Are we there yet?" *Empirical Software Engineering*, vol. 28, no. 2, p. 45, 2023.
- [18] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [19] T. Y. Zhuo, "ICE-Score: Instructing large language models to evaluate code," in *Findings of the Association for Computational Linguistics: EACL 2024*. Association for Computational Linguistics, 2024, pp. 2232–2242. [Online]. Available: <https://aclanthology.org/2024.findings-eacl.148/>
- [20] P. Verga, S. Hofstätter, S. Althammer, Y. Su, A. Piktus, A. Arkhangorodsky, M. Xu, N. White, and P. Lewis, "Replacing judges with juries: Evaluating LLM generations with a panel of diverse models," 2024. [Online]. Available: <https://arxiv.org/abs/2404.18796>
- [21] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [22] B. Lin, S. Scalabrino, A. Mocci, R. Oliveto, G. Bavota, and M. Lanza, "Investigating the use of code analysis and NLP to promote a consistent usage of identifiers," in *Proceedings of the 17th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM 2017)*. IEEE, 2017, pp. 81–90.
- [23] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [24] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1711.00740>
- [25] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 2023, pp. 31–53.
- [26] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li, F. Luo, Y. Xiong, and W. Liang, "DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence," 2024. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [27] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-Coder technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2409.12186>
- [28] J. Cordeiro, E. Noei, and Y. Zou, "An empirical study on the code refactoring capability of large language models," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [29] Z. Wang, L. Zhang, C. Cao, N. Luo, X. Luo, and P. Liu, "How does naming affect language models on code analysis tasks?" *Journal of Software Engineering and Applications*, vol. 17, no. 11, pp. 803–816, 2024.
- [30] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [31] R. Ravi, D. Bradshaw, S. Ruberto, G. Jahangirova, and V. Terragni, "LLMLOOP: Improving LLM-generated code and tests through automated iterative feedback loops," in *Proceedings of the 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025.
- [32] A. Hosna, E. Merry, J. Gyalmo, Z. Alom, Z. Aung, and M. A. Azim, "Transfer learning: A friendly introduction," *Journal of Big Data*, vol. 9, no. 1, p. 102, 2022.
- [33] K. W. Church, Z. Chen, and Y. Ma, "Emerging trends: A gentle introduction to fine-tuning," *Natural Language Engineering*, vol. 27, no. 6, pp. 763–778, 2021.
- [34] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Singapore: Association for Computational Linguistics, 2023, pp. 2511–2522. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.153/>

- [35] R. Bavishi, M. Pradel, and K. Sen, "Context2Name: A deep learning-based approach to infer natural variable names from usage contexts," 2018. [Online]. Available: <https://arxiv.org/abs/1809.05193>
- [36] S. Scalabrino, M. Linares-Vásquez, R. Oliveto, and D. Poshyvanyk, "A comprehensive model for code readability," *Journal of Software: Evolution and Process*, vol. 30, no. 6, p. e1958, 2018.
- [37] T. Takebayashi, A. Peruma, M. W. Mkaouer, and C. D. Newman, "An exploratory study on the usage and readability of messages within assertion methods of test cases," 2023.
- [38] J. Jiang, J. Shen, S. Kim, K. M. Yoo, J. Kim, and S. Kim, "ReflexiCoder: Teaching large language models to self-reflect on generated code and self-correct it via reinforcement learning," 2026. [Online]. Available: <https://arxiv.org/abs/2603.05863>
- [39] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [40] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [41] N. Jain, P.-y. Chiang, Y. Wen, J. Kirchenbauer, H.-M. Chu, G. Somepalli, B. R. Bartoldson, B. Kailkhura, A. Schwarzschild, A. Saha, M. Goldblum, J. Geiping, and T. Goldstein, "NEFTune: Noisy embeddings improve instruction finetuning," in *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: <https://openreview.net/forum?id=0bMmZ3fkCK>
- [42] J. Liu, "codereader: A tool for grading unit test readability using LLMs," <https://pypi.org/project/codereader/>, 2025, accessed: 2025.
- [43] C. Callison-Burch, M. Osborne, and P. Koehn, "Re-evaluating the role of BLEU in machine translation research," in *Proceedings of the 11th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. Trento, Italy: Association for Computational Linguistics, 2006, pp. 249–256. [Online]. Available: <https://aclanthology.org/E06-1032/>
- [44] M. Tufano, S. K. Deng, N. Sundaresan, and A. Svyatkovskiy, "Methods2test: A dataset of focal methods mapped to test cases," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ser. MSR '22. ACM, May 2022.
- [45] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.
- [46] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. ACM, 2011, pp. 1–10.
- [47] OpenAI, "Introducing GPT-5," <https://openai.com/index/introducing-gpt-5/>, 2025, accessed May 2025.
- [48] H. Liu, Y. Wang, Z. Wei, Y. Xu, J. Wang, H. Li, and R. Ji, "RefBERT: A two-stage pre-trained framework for automatic rename refactoring," 2023.
- [49] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating sequences from structured representations of code," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=H1gKY09tX>
- [50] W. A. M. C. Ouédraogo, K. Li *et al.*, "Human-aligned code readability assessment with large language models," 2025.
- [51] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [52] I. Olkin and J. W. Pratt, "Unbiased estimation of certain correlation coefficients," *Annals of Mathematical Statistics*, vol. 29, no. 1, pp. 201–211, 1958.
- [53] R. A. Fisher, "Frequency distribution of the values of the correlation coefficient in samples from an indefinitely large population," *Biometrika*, vol. 10, no. 4, pp. 507–521, 1915.
- [54] Vast.ai, "Vast.ai: Gpu cloud computing platform," <https://vast.ai>, 2024, accessed May 2025.

APPENDIX

The pipeline uses three prompt templates. Prompt 1 is sent as the system instruction. Prompt 2 is the user-facing prompt.

Prompt 3 is the retry prompt, invoked when a response fails schema validation following the Reflexion framework [30].

```
You are a Java test code refactoring expert.
You will be given:
- A Java test method (wrapped in a dummy class), and
- A list of identifiers to rename, each tagged with
  its kind: (method) or (variable).

Your job is to propose meaningful names for ALL of
the listed identifiers.

Naming conventions to follow:
- Methods (func_*): use camelCase starting with '
  test',
  e.g. testShouldReturnNullWhenInputIsEmpty.
- Variables used in assertions (var_*): prefer
  'expected' / 'actual' where applicable.
- Other variables (var_*): use concise camelCase
  nouns
  that describe the value, e.g. userCount,
  resultList.

Rules:
- You MUST include every identifier from the list as
  a key -- no omissions.
- Use ONLY the listed identifiers as keys -- no
  extras.
- You MUST ONLY respond with a valid JSON object
  mapping originalName -> newName.
- Do NOT output code, comments, markdown, or any
  text
  outside the JSON object.
```

Listing 1. System Instruction Prompt

```
Here is the obfuscated Java test method wrapped in a
dummy class:

'''java
{test_case}
'''

Here are ALL the identifiers that must be renamed:
{identifiers}

Rename ALL of the identifiers above. You MUST
include
every one of them as a key.
Return a single JSON object mapping originalName ->
newName.

Example (for a method + two variables):
{"func_1": "testShould...",
 "var_1": "actualResult",
 "var_2": "expectedSize"}

Rules:
- ALL listed identifiers must appear as keys.
- Use ONLY the listed identifiers as keys.
- Do NOT output anything except the JSON object.
```

Listing 2. User Prompt Template

```
Here is the original obfuscated Java test method
wrapped in a dummy class:

'''java
{test_case}
'''

Here are ALL the identifiers that must be renamed:
{identifiers}
```


Your previous response was rejected for this reason:
{error_reason}

Your previous (rejected) response was:
{failed_response}

Please try again. Make sure your new response:
- Includes EVERY identifier from the list as a key.
- Uses ONLY the listed identifiers as keys.
- Is a valid JSON object only -- no backticks, no explanation.

Listing 3. Retry Prompt Template

The CodeReader configuration YAML defines the scoring rubric and model ensemble.

```
language: java

settings:
  timeout_seconds: 320
  health_timeout_seconds: 900
  max_concurrency: 1
  fail_on_no_active_models: true
  log_path: out/temp/readability_results.txt
  debug_jsonl_path: responses.jsonl

scoring:
  scale: 0-100
  focus: identifier_naming
  weights:
    identifier_naming: 0.75
    assertion_quality: 0.10
    test_independence: 0
    behavioral_specificity: 0.15

rules:
  - Clarity of variable and method names: assess whether identifier names clearly convey intent without needing to read the full test body. Penalise generic names like var1, obj, result0, func_1 or single letters unless used as counters.
  - Renamed vs. generic baseline: explicitly penalise identifiers that appear to still be auto-generated (e.g. var0, obj1, int0, string1, list0). These represent failures of the renaming step.
  - Cross-identifier consistency within a single test.
  - Alignment with actual test logic.
  - Test method naming and behavioral specificity.
  - Assertion quality and magic numbers.
  - Test independence (weight 0 in this config).
  - Ignore enclosing class name.
```

Listing 4. CodeReader Configuration (Part 1 – Scoring & Rules)

```
models:
  - name: deepseek-coder:6.7b
    weight: 1.4
    provider: ollama
  - name: qwen3.5:9b
    weight: 1.25
    provider: ollama
  - name: phi3:medium
    weight: 0.75
    provider: ollama
```

Listing 5. CodeReader Configuration (Part 2 – Model Ensemble)

```
@Test(timeout = 4000)
public void test42() throws Throwable {
```

```
Calculator calculator0 = new Calculator();
double double0 = calculator0.modulo(
    1.0, 1408.0511499512309);
assertEquals(1, calculator0.getOperationCount());
;
assertEquals(1.0, double0, 0.01);
}
```

Listing 6. Example EvoSuite-generated test before renaming

```
@Test
public void test011() throws Throwable {
    if (debug)
        System.out.format("%n%s%n",
            "RegressionTest0.test011");
    Calculator calculator0 = new Calculator();
    calculator0.reset();
    java.lang.String str3 =
        calculator0.classify((double) (byte)10);
    calculator0.memoryStore(0.0d);
    calculator0.memoryClear();
    double double9 = calculator0.max(
        (double)10.0f, (-1.0d));
    java.lang.Class<?> wildcardClass10 =
        calculator0.getClass();
    org.junit.Assert.assertEquals(str3, "medium");
    org.junit.Assert.assertTrue(double9 == 10.0d);
    org.junit.Assert.assertNotNull(wildcardClass10);
}
```

Listing 7. Example Randoop-generated test before renaming

The dataset is constructed by obfuscating human-written test code to simulate the generic identifier naming produced by automated test generation tools.

```
public class TestClass10000 {
    @Test public void func_1() {
        Map<Integer, String> var_1 =
            new HashMap<Integer, String>();
        var_1.put(1, "one");
        var_1.put(2, "two");
        var_1.put(3, "three");
        var_1.put(4, "four");
        Assert.assertEquals(
            "{_1->_one_,_2->_two_,_3->_three_,_4->_four_}",
            MapUtils.toString(var_1));
    }
}
```

Listing 8. Example obfuscated test case

```
{
  "prompt": "public_class_TestClass10000_{\n\n\n\n@Test_public_void_func_1()_{...}_}\n",
  "response": "public_class_TestClass10000_{\n\n\n\n@Test_public_void_shouldPrettyPrintMap()_{\n\n\n\nMap<Integer,_String>_map=_..._}\n}"
```

Listing 9. Example JSONL dataset sample

Effect sizes are computed using the Wilcoxon signed-rank test on paired observations, following the non-parametric approach recommended for software engineering experiments with non-normal distributions [46]. The effect-size statistic is $r = Z/\sqrt{N}$, interpreted as small ($|r| \geq 0.10$), medium ($|r| \geq 0.30$), or large ($|r| \geq 0.50$) following Cohen [45]. To correct for small-sample inflation, r is bias-adjusted using the Olkin–Pratt estimator [52]. The 95% confidence interval

TABLE IV
RECOMMENDED n FOR PRIMARY COMPARISONS. ALL LARGE-EFFECT
COMPARISONS ARE WITHIN THE 100-SAMPLE BUDGET.

Comparison	0%	15%	25%	50%	75%	100%
F1 vs zero baseline	2	2	2	2	2	2
LLM avg: obf vs renamed	2	4	4	3	7	3
LLM wavg: obf vs renamed	3	5	5	4	10	3
LLM avg: obf vs oracle	2	3	2	3	3	2
LLM wavg: obf vs oracle	3	3	3	4	4	3

TABLE V
FULL BENCHMARK METRICS PER CHECKPOINT. EM-ORD = ORDERED
EXACT MATCH, EM-UNORD = UNORDERED EXACT MATCH, CER =
CHARACTER ERROR RATE.

Ckpt	P	R	F1	EM-ord	EM-unord	CER	Oracle	Oracle (w)
0%	0.714	0.817	0.760	0.753	0.730	32.03	68.13	71.11
15%	0.848	0.825	0.834	0.785	0.792	17.53	67.89	70.68
25%	0.853	0.831	0.839	0.793	0.804	16.94	68.21	71.15
50%	0.849	0.828	0.836	0.785	0.801	17.54	68.13	70.45
75%	0.853	0.824	0.836	0.785	0.798	17.18	67.31	70.09
100%	0.849	0.826	0.835	0.785	0.793	17.27	68.43	70.90

on r is obtained via the Fisher Z -transformation [53]. The recommended sample size n is derived from the conservative lower bound of this confidence interval at $\alpha = 0.05$, power = 0.80, with a 25% non-normality correction.

For all primary comparisons (obfuscated vs. renamed and obfuscated vs. oracle), the recommended n never exceeds 10 across all checkpoints. The most demanding comparison, renamed vs. oracle at the 0% checkpoint, requires up to 2085 samples; however, this reflects a small, marginally significant effect ($r = -0.23$, $p = 0.048$) whose weighted-average variant is not significant ($p = 0.20$), indicating inconsistent rather than systematically poor output. At the observed throughput of 3–6 hours per 100 samples, scaling to the maximum recommended n is infeasible within the hardware and time constraints of this work.

All fine-tuning and inference experiments for Qwen2.5-Coder were conducted on cloud computing infrastructure via VastAI [54] on an NVIDIA 5090 GPU with 32 GB of GPU memory. The CodeReader judge ensemble was served together on the cloud via Ollama during evaluation.

TABLE VI
TOKEN USAGE AND LATENCY STATISTICS PER CHECKPOINT.

Ckpt	Prompt	Completion	Total	Latency (ms)	Success
0%	53,053	4,779	57,832	2,940	100/100
15%	53,053	4,333	57,386	7,582	100/100
25%	53,053	4,589	57,642	8,274	100/100
50%	53,053	4,387	57,440	7,044	100/100
75%	53,053	4,488	57,541	9,943	100/100
100%	53,053	4,542	57,595	4,779	100/100